Python Basics Notes

Prapti Babasaheb Gagare
Roll Number: MS2504

1 Variables

- Variables is a name in python refers to the value stored in memory.
- Value is stored once and used multiple times.
- Easy to change a value in place instead of changing everywhere.
- Variables are used to store user input.
- Making decisions.

Syntax

```
variable_name = value
```

2 Basic Data Types

Data type tells us which type of value is stored in a variable.

a. Integer

- It refers to the number without decimal point.
- Integer is used when we have to represent countable things (Age, Roll_no).

b. Float

- Float is a number with decimal point.
- Some values are not in integers like height, weight; then we use float.

c. Boolean

- Boolean has only 2 values: True and False.
- It is mostly used in decision making.

d. String

- String is a sequence of characters.
- String is used when text values are declared to variables (e.g., Name, email).

3 Arithmetic Operators

- These are symbols which perform mathematical operation on numbers.
- These operators are used to perform calculation.
- Arithmetic operators are addition (`+`), subtraction (`-`), multiplication (`*`), division (`/`), floor division (`//`), exponent (`**`), modulus (`%`).

Syntax

```
operand1 operator operand2
```

4 Comparison Operators

- These are the operators that compare two values and return the result in the form of Boolean values.
- They are mainly used in decision making.
- Comparison operators are Equal to (`==`), Not equal to (`!=`), greater than (`>`), less than (`<`), greater than equal to (`>=`), less than equal to (`<=`).

Syntax

```
operand1 operator operand2
```

5 Assignment Operators

- These are the operators that assign a value to a variable.
- The assignment operators are Assignment (`=`), Add and assign (`+=`), Subtract and assign (`-=`), Multiply and assign (`*=`), Divide and assign (`/=`), Floor divide and assign (`//=`), Exponent and assign (`**=`), Modulus and assign (`%=`).

Syntax

```
operand1 operator operand2
```

6 Type Conversion

- This is the process of converting a value from one data type to another.
- We use type conversion because different data types cannot always be used together.
- For type conversion python provides built-in functions: for Integer it is `int()`, for String it is `str()`, for Float it is `float()`, and for Boolean it is `bool()`.

7 Input and Output

- Input and output functions are used to accept data from user and display it.
- `input()` – accepts data from user.
- `print()` – displays data to user.

8 Strings

- String is a sequence of characters enclosed in quotes.
- String can contain letters, symbols, numbers.

String Creation

- String creation is the process of creating a string value and storing it in a variable so later we can manipulate the text data in a program.
- It is created by placing characters in single quotes (`' '`), double quotes (`" "`), and triple quotes (`""" """` or `''' '''`).

Indexing

- In this we access the individual element of a string using their position.
- It is used when we have to work with a specific character in a string.

Syntax

```
string[index]
```

Slicing

- In this a substring is extracted from a string using an index range.
- Index can access a single character but slicing can access multiple characters.

Syntax

```
string[start_index:end_index]
```

String Operations

- String concatenation (`+`) – joins 2 or more strings.
- String repetition (`*`) – repeats a string multiple times. Mostly used in creating patterns.

There are also some string functions to convert string to lowercase (`lower()`), uppercase (`upper()`), and finding string length (`len()`).

9 Lists

- A list is an ordered collection of multiple values.
- A list can store strings, numbers, and different data types together.

Creating a List

List is created using square brackets `[]` or the `list()` function.

Indexing

- It is similar to string indexing, accessing the element based on their position.
- Without indexing we cannot access a specific value from a list.

Syntax

```
list_name[index]
```

Slicing

- Here the sublist from a list is accessed based on their position using a range.
- Indexing accesses one element while slicing accesses multiple elements.

Updating Elements

- Updating elements in a list means changing or replacing the existing value of an element with a new value.
- Because lists are mutable, we can modify list elements after creating a list.

Nested Lists

- Nested list means a list within a list, i.e., the list has elements in the form of lists.
- We need nested lists mainly when the data is in tabular form.

Syntax

```
list = [[list1], [list2], [list3]]
```

Basic List Methods

a. **append()**

- Method used to add a new element at the end of the list.
- Used when we want to add new data in an existing list without creating a new list.

Syntax

```
list_name.append(value)
```

b. **insert()**

- When we want to add data at a particular location we use `insert()`.

Syntax

```
list_name.insert(index, value)
```

c. **remove()**

- Used to delete the first occurrence of a specific value from a list.

Syntax

```
list_name.remove(value)
```

d. **pop()**

- Used to remove and return an element from a list using its index; if index is not given then it removes the last element.

Syntax

```
list.pop(index)
```

e. **sort()**

- Used to arrange elements of a list in ascending or descending order.

Syntax

```
list.sort()
```

f. **reverse()**

- Used to reverse the order of elements in a list.

Syntax

```
list.reverse()
```

g. **len()**

- Used to find the number of elements in a list.

Syntax

```
len(list)
```

10 Mini Experiments

GitHub link: https://github.com/Prapti-gagare/Python

11 List, Tuple, Set, and Dictionary Methods

1. List

A list is an ordered, mutable (changeable) collection of items that can hold elements of any data type, including duplicates. Lists are defined using square brackets `[]`.

a. **append()**

- What: Adds a single new item to the end of the list.
- Why: Used when you need to grow a list one element at a time, such as collecting results in a loop.
- How: Call the method on the list and pass the item to add; the list is modified in place and nothing is returned.

Syntax

```
list.append(item)
```

b. **insert()**

- What: Places a new item at a specific position in the list instead of at the end.
- Why: Used when the order of elements matters and a new item must appear at a particular location.
- How: Pass the index where the item should go, followed by the item itself; existing elements shift right to make room.

Syntax

```
list.insert(index, item)
```

c. **remove()**

- What: Deletes the first occurrence of a specified element from the list.
- Why: Used when you know the value you want gone but not its index.
- How: Call the method with the value to remove; Python searches the list and removes only the first match (raises an error if not found).

Syntax

```
list.remove(item)
```

d. **pop()**

- What: Removes an element from the list and returns it.
- Why: Used when you need to take an item out of the list and still use its value afterward, such as processing items off a stack.
- How: Call with an index to remove that specific element, or call with no argument to remove and return the last element.

Syntax

```
list.pop(index)
```

e. **sort()**

- What: Rearranges the elements of the list in place.
- Why: Used when you need the list ordered, such as displaying items alphabetically or numerically.
- How: Call the method with no arguments for ascending order by default; the original list is modified directly.

Syntax

```
list.sort()
```

2. Tuple

A tuple is an ordered, immutable (unchangeable) collection of items that can hold elements of any data type, including duplicates. Tuples are defined using parentheses `()`.

a. **count()**

- What: Counts how many times a specified element appears in the tuple.
- Why: Used when you need to know the frequency of a value without writing a manual loop.
- How: Call the method on the tuple with the value to count; it returns an integer.

Syntax

```
tuple.count(item)
```

b. **index()**

- What: Finds the position of the first occurrence of a specified element.
- Why: Used when you need to know where a value sits inside the tuple.

- How: Call the method with the value to search for; it returns the index (raises an error if the value isn't present).

Syntax

```
tuple.index(item)
```

c. **len()**

- What: Returns the total number of elements in the tuple.
- Why: Used when you need to know the size of the tuple, for example to control loop ranges.
- How: Pass the tuple as an argument to the built-in `len()` function; it returns an integer count.

Syntax

```
len(tuple)
```

d. **max()**

- What: Finds the largest element in the tuple.
- Why: Used when comparing numeric or orderable values to find the highest one.
- How: Pass the tuple as an argument to the built-in `max()` function; it returns the largest value found.

Syntax

```
max(tuple)
```

e. **min()**

- What: Finds the smallest element in the tuple.
- Why: Used when comparing numeric or orderable values to find the lowest one.
- How: Pass the tuple as an argument to the built-in `min()` function; it returns the smallest value found.

Syntax

```
min(tuple)
```

3. Set

A set is an unordered, mutable collection of unique items with no duplicate values. Sets are defined using curly braces {}.

a. **add()**

- What: Inserts a new unique element into the set.
- Why: Used when you want to grow a set while automatically avoiding duplicate values.
- How: Call the method with the item to add; if the item already exists, the set stays unchanged.

Syntax

```
set.add(item)
```

b. **discard()**

- What: Removes an element from the set if it is present.
- Why: Used for safe removal, when you're not sure whether the item exists and don't want an error.
- How: Call the method with the value to remove; unlike `remove()`, it does nothing if the value is absent.

Syntax

```
set.discard(item)
```

c. **update()**

- What: Adds all elements from another set into the current set.
- Why: Used when merging two sets together while keeping only unique values.
- How: Call the method with another set (or iterable) as the argument; the current set is modified in place.

Syntax

```
set.update(other_set)
```

d. **union()**

- What: Combines two sets into a new set containing all unique elements from both.
- Why: Used when you need the combined result but want to keep the original sets untouched.
- How: Call the method on one set, passing the other set as the argument; it returns a brand-new set.

Syntax

```
set.union(other_set)
```

e. **intersection()**

- What: Finds the elements that are common to both sets.
- Why: Used when you need to know what two sets share in common.
- How: Call the method on one set, passing the other set as the argument; it returns a new set of shared elements.

Syntax

```
set.intersection(other_set)
```

4. Dictionary

A dictionary is an unordered, mutable collection of key-value pairs, where each key must be unique. Dictionaries are defined using curly braces {} with `key:value` syntax.

a. **get()**

- What: Retrieves the value associated with a specified key.
- Why: Used to safely read a value without risking an error if the key doesn't exist (it returns `None` instead).
- How: Call the method with the key to look up; it returns the matching value or `None` if absent.

Syntax

```
dict.get(key)
```

b. **keys()**

- What: Returns a view of all the keys in the dictionary.
- Why: Used when you need to see or iterate over just the keys, for example to check membership.
- How: Call the method with no arguments; it returns a `dict_keys` view object.

Syntax

```
dict.keys()
```

c. **values()**

- What: Returns a view of all the values in the dictionary.
- Why: Used when you need to work with just the values, for example to sum or search them.
- How: Call the method with no arguments; it returns a `dict_values` view object.

Syntax

```
dict.values()
```

d. **items()**

- What: Returns all key-value pairs in the dictionary as tuples.
- Why: Used most often to loop through both keys and values together at the same time.
- How: Call the method with no arguments; it returns a `dict_items` view of (key, value) tuples.

Syntax

```
dict.items()
```

e. **update()**

- What: Updates the dictionary with new key-value pairs, adding new keys and overwriting existing ones.
- Why: Used when merging new data into an existing dictionary in one step, rather than assigning each key one by one.
- How: Call the method with another dictionary (or key-value pairs); matching keys get overwritten and new keys get added.

Syntax

```
dict.update(key:  value)
```